Week 8 · Git-flow, and organising the code

Technical Interaction Design · ITU · Autumn 2026

Built 8 November 2026 · commit `55039c3`
Latest version: https://itu-tid.github.io/lecture-notes-pdf/Week-08.pdf

CONTENTS

Team Collaboration with Git Flow

- Until now each of you had your own app; from this week the group shares one
- The repo lives in an organisation the group owns, not in one student's account
- Git Flow gives every branch one job
- A feature branch is where you are allowed to break things
- Nothing reaches `development` without somebody else reading it first
- `main` contains only releases, and only the staff can change it
- Neither `main` nor `development` is ever broken
- `node_modules` stays out of the repo, through `.gitignore`

Pulling Out the Service Layer

- Every component in todo-26 knows how Parse works
- A component says *what* should happen; a service says *how*
- The service knows nothing about React; the state stays in the component
- The service throws; the component decides how to tell the user
- Move one function at a time, and click through the app after each move
- When it is done, only `services/` imports Parse
- The payoff is a single place to change when the backend does

Code Organization

- A flat `src/` works until you have to look for things
- One folder per kind of file: pages, components, services
- A component made of several files gets a folder of its own
- A file has the name of the component it exports
- Moving files is a refactoring too, so do it on a branch of its own
- No organisation is final

Component Extraction Cheat Sheet

- Every pull request is judged against the same five questions
- Extract a block when it repeats, or when you can name it and the page reads better without it
- Four tests catch most bad extractions
- An extraction moves JSX, and whatever it used from outside becomes a prop
- The same mistakes show up in every review
- For a deeper dive

Finding the Components

- Start by drawing boxes on the app you can already see
- Two boxes if either half would be useful on its own
- Extracting a component is moving JSX out and naming what it needs

Team Collaboration with Git Flow

Source: Lectures/Technical/Tooling/Collaboration-with-Git-Flow.md — something unclear? Open an issue, or send a pull request.

Until now each of you had your own app; from this week the group shares one

Up to now, every one of you wrote your own to-do app, alone, in your own repo. Nobody else pushed to it, so nobody else could break it, and nobody else had to understand it.

That ends this week. Your group's project starts from one app:

1. **Choose one**: one of yours, or `todo-26` as it is at `week-08-components`.
2. **Make it the group's**: put it in a GitHub organisation of your own (below).
3. **Clean it up**: pull out the service layer, find the components, as in this week's notes.
4. **Then adapt it** to the app your group is actually building.

From step 3 onwards, four or five people are changing the same code. Git already lets you do that. What it does not give you is an agreement about *how*, and that is what this note is about.

Something unclear in this section? Open an issue.

The repo lives in an organisation the group owns, not in one student's account

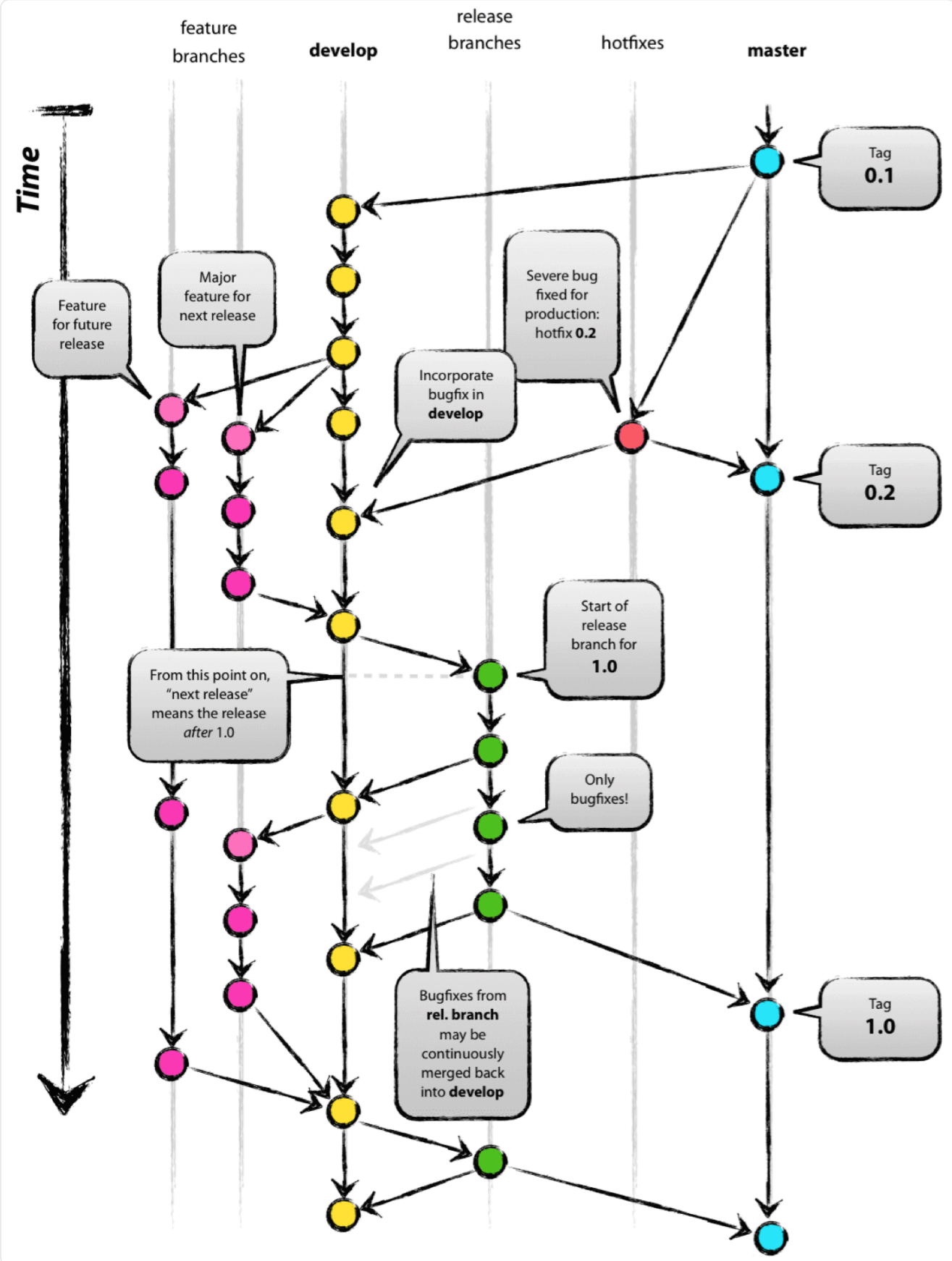
Create a GitHub organisation for the group (it is free), and put the repo there. Invite everybody in the group as an owner, and invite the TID staff as well.

The alternative, which every cohort used to pick, is one student's personal repo with the others added as collaborators. It works, technically. But it is *Ada's* repo, with Ada's name in every URL, and the others feel like guests in it. In an organisation the project belongs to the group, and so does the responsibility for it.

Something unclear in this section? Open an issue.

Git Flow gives every branch one job

Git Flow is a popular way of choreographing collaboration with Git, built around several kinds of branches, each with one job:



The picture uses the original names, `develop` and `master`. In TID we use a simplified version, with today's names, and three kinds of branch:

- **`main`**: releases, the versions you hand in
- **`development`**: the group's latest work, integrated
- **feature branches**: one per piece of work in progress, one person or a pair each

The rest of this note is what each of them is for.

Something unclear in this section? Open an issue.

A feature branch is where you are allowed to break things

When you start something (a new screen, a bug fix, pulling `shareList` into a service), branch off `development`:

```
git checkout development
git pull
git checkout -b share-list-service
```

Commit as often as you like. Half-finished, broken, experimental: nobody else is standing on this branch, so nobody else falls over when it breaks. That is the whole point of a branch. You break your own branch, not the one four other people are standing on.

Something unclear in this section? Open an issue.

Nothing reaches `development` without somebody else reading it first

A finished feature goes to `development` through a **pull request**, and you tag a teammate to review it. Only after they have read it, and are happy with it, is it merged.

This is the part that matters, far more than any branch name. A review is a **quality gate**: a point that code cannot pass without a second person checking it. Two things come out of every one:

- **You get feedback**, from somebody who did not write it and so does not know what you *meant*, only what you wrote.
- **Two people know the code** instead of one. When you are ill in the week before the hand-in, somebody else can fix it.

Small pull requests get real reviews. A pull request that moves one function into a service can be read in two minutes. One that touches every file gets a ✅ from someone who scrolled past it.

Something unclear in this section? Open an issue.

`main` contains only releases, and only the staff can change it

When `development` is in a state you would hand in, it goes to `main`, again through a pull request. But this one you do not merge yourselves: **only the TID staff can accept pull requests into `main`.** That is our quality gate, the same idea one level up.

How to set that up is left to you. GitHub calls it *branch protection*, and the settings move around every year, so look it up. What we care about is that you understand why it is there, not which checkbox does it.

Something unclear in this section? Open an issue.

Neither `main` nor `development` is ever broken

Every cohort has had at least one team whose `main` did not compile, usually for days, usually because a branch was merged in a hurry near a deadline.

It matters more than it looks. `development` is what every teammate branches from; `main` is what the examiner clones and what your deployed app is built from. A broken shared branch is not one person's problem for an afternoon. It is everybody's problem until somebody notices.

So: pull, build, and run *before* you merge, not after. A feature branch can be broken. The branches other people stand on cannot.

Something unclear in this section? Open an issue.

`node_modules` stays out of the repo, through `.gitignore`

`.gitignore`, in the root of the repo, lists the files Git should not track. The one that matters most is `node_modules`: hundreds of megabytes that anybody can recreate with `npm install`, and that differ from one laptop to the next. Vite's starter already ignores it. Check that yours still does before the first push to the group's repo.

Your `.env.local`, with your Back4App keys, belongs in there too. `todo-26` ignores both; if your group starts from somebody else's app, check.

Something unclear in this section? Open an issue.

Exam Questions

1. Describe the three kinds of branches used in the course's Git Flow model, and what each one is for.

2. What is a quality gate, and where are the two quality gates in the course's Git Flow model?

3. Why should `node_modules` be in `.gitignore`?

Pulling Out the Service Layer

Source: Lectures/Technical/Structure/Pulling-Out-the-Service-Layer.md — something unclear? Open an issue, or send a pull request.

It works on the `todo-26` app as it stands after Routing in React. You will not write a single new query in this note. Every query is already there; it only moves.

> **In todo-26:** `git checkout week-06-routing` is where this note starts, and `git checkout week-08-service-layer` is where it ends. `git diff week-06-routing week-08-service-layer -- src` shows every line that moved.

Every component in todo-26 knows how Parse works

Count the files in `src/` that `import Parse from "parse"`: `App.jsx`, `ToDoList.jsx`, `ListPage.jsx`, `AuthPage.jsx`. Half of the eight components. And `Home`, which does not import it, still calls `item.get("name")` on a Parse object it was handed.

The clearest case is sharing a list. Here is what `App` runs when somebody clicks *Share*:

```
async function handleShare(list, username) {
  try {
    const friend = await new Parse.Query(Parse.User)
      .equalTo("username", username)
      .first();

    if (!friend) {
      alert("No user with that name");
      return;
    }

    const acl = list.getACL() ?? new Parse.ACL(Parse.User.current());
    acl.setReadAccess(friend, true);
    list.setACL(acl);

    const todos = await new Parse.Query(TodoItem).equalTo("list", list).find();
    todos.forEach((todo) => todo.setACL(acl));

    const member = new ListMember();
    member.set("list", list);
    member.set("user", friend);
    // … the member's own ACL …

    await Parse.Object.saveAll([list, ...todos, member]);

    alert(`Shared ${list.get("name")} with ${username}`);
  } catch (error) {
    alert(error.message);
  }
}
```

Forty lines in a click handler, and only the two `alert`s are about the screen. The rest is about ACLs, pointers and which rows to save together. `App` is the component that decides which page to draw, and it also knows that sharing a list means rewriting the ACL of every to-do in it.

Nothing is broken. It worked when each of you wrote it alone. It stops working when four people share the code: the one changing how the home page looks and the one fixing a sharing bug are editing the same function.

Something unclear in this section? Open an issue.

A component says *what* should happen; a service says *how*

This is the principle from Refactoring by Extracting Components, one level up: **code in a component should be at the same abstraction level.** A click handler that reads "share this list with this user, then tell the user it worked" is at one level. One that builds queries and ACLs between those two steps is at three.

So the *how* goes into a **service**: a plain JavaScript module of `async` functions, named in the language of the app rather than the language of the database. `shareList(list, username)`, not `saveAllWithAcl(...)`. The component calls it and handles the outcome.

The services live in their own folder, as in Code Organization:

```
src/
├── services/
│   ├── parse.js          ← Parse.initialize, and nothing else
│   ├── authService.js    ← signUp, logIn, logOut, currentUser
│   ├── listService.js    ← loadMyLists, loadSharedWithMe, loadList, addList, shareList,
leaveList
│   └── todoService.js    ← loadTodos, addTodo, deleteTodo, setDone
├── App.jsx
├── ToDoList.jsx
└── …
```

One service per kind of thing the app handles, which in todo-26 is one per Parse class, give or take: users, lists, to-dos.

Something unclear in this section? Open an issue.

The service knows nothing about React; the state stays in the component

Here is `todoService.js`, every line of it moved out of `ToDoList.jsx`:

```
import Parse from "./parse.js";

const TodoItem = Parse.Object.extend("TodoItem");

// may the current user change this object? asks its ACL
export function canWrite(parseObject) {
  const acl = parseObject.getACL();
  if (!acl) return true; // no ACL at all: everybody may write

  const user = Parse.User.current();
  return acl.getPublicWriteAccess() || acl.getWriteAccess(user);
}

// a row from the database, as the components want to see it
function toTodo(item) {
  return {
    id: item.id,
    text: item.get("text"),
    done: item.get("done"),
    canWrite: canWrite(item),
  };
}

export async function loadTodos(list) {
  const results = await new Parse.Query(TodoItem)
    .equalTo("list", list)
    .ascending("createdAt")
    .find();

  return results.map(toTodo);
}

export async function addTodo(list, text) {
  const item = new TodoItem();
  item.set("text", text);
  item.set("done", false);
  item.set("list", list);

  // the to-do follows its list: whoever may read the list may read the to-do
  item.setACL(list.getACL());

  return toTodo(await item.save());
}
```

```
export async function deleteTodo(id) {
  // we only have the id, so we build a stand-in object pointing at that row
  await TodoItem.createWithoutData(id).destroy();
}

export async function setDone(id, done) {
  const item = TodoItem.createWithoutData(id);
  item.set("done", done);
  await item.save();
}
```

No `useState`, no JSX, no `alert`. It takes arguments, talks to the server, and returns data or throws. You could call it from a test, from a script, or from a different UI altogether.

`ToDoList` keeps its `todos` state, and keeps deciding what to do with the answer:

```
import {
  loadTodos,
  addTodo,
  deleteTodo,
  setDone,
  canWrite,
} from "./services/todoService.js";

export default function ToDoList({ list }) {
  const [todos, setTodos] = useState([]);

  useEffect(() => {
    loadTodos(list)
      .then(setTodos)
      .catch((error) => alert(error.message));
  }, [list]);

  async function handleAdd(text) {
    try {
      const todo = await addTodo(list, text);
      setTodos([...todos, todo]);
    } catch (error) {
      alert(error.message);
    }
  }

  async function handleDelete(id) {
    try {
      await deleteTodo(id);
      setTodos(todos.filter((each) => each.id !== id));
    } catch (error) {
      alert(error.message);
    }
  }

  async function handleToggle(id) {
    const todo = todos.find((each) => each.id === id);
    try {
      await setDone(id, !todo.done);
      setTodos(todos.map((t) => (t.id === id ? { ...t, done: !t.done } : t)));
    } catch (error) {
      alert(error.message);
    }
```

```
  }

  // … the same JSX as before …
}
```

Each handler now has the same three beats: ask the service, update the state, or tell the user it failed. You can read it without knowing what Parse is.

Something unclear in this section? Open an issue.

The service throws; the component decides how to tell the user

Look at what happened to the two `alert`s in `handleShare`. The service cannot call `alert`. It does not know whether it is running behind a button, a form with an error line, or a test. So where the old code alerted and returned, the service throws:

```
// listService.js
export async function shareList(list, username) {
  const friend = await new Parse.Query(Parse.User)
    .equalTo("username", username)
    .first(); // undefined if nobody has that username

  if (!friend) {
    throw new Error("No user with that name");
  }

  // … the ACLs, the ListMember row, the saveAll: moved here unchanged …
}
```

and `App` is left with the part that is about the screen:

```
async function handleShare(list, username) {
  try {
    await shareList(list, username);
    alert(`Shared ${list.get("name")} with ${username}`);
  } catch (error) {
    alert(error.message);
  }
}
```

The user sees exactly what they saw before. That is the test of a refactoring: **the behaviour does not change, only where the code lives.**

Something unclear in this section? Open an issue.

Move one function at a time, and click through the app after each move

Do not move everything in one sitting and then find out what broke. Move one function, run the app, try the thing it does, commit. Then the next.

A good order is from the leaves inwards:

1. `parse.js`: move `Parse.initialize` and the credentials check out of `App.jsx`. Every other service imports `Parse` from here, so it is always initialised first.
2. `ListPage`'s single query becomes `loadList(listId)`. One function, easy to check.
3. `ToDoList`'s four operations become `todoService.js`, as above.
4. `AuthPage`'s sign-up and login, and `App`'s logout, become `authService.js`.
5. `App`'s lists, sharing and leaving become `listService.js`, with `shareList` last, because it is the biggest.

Each step is one branch and one pull request, which is what Collaboration with Git-Flow is for. A reviewer can check "this moved, and nothing else changed" in a two-minute diff. Nobody can check that in a diff that touches every file.

Something unclear in this section? Open an issue.

When it is done, only `services/` imports Parse

There is a one-line check that the job is finished:

```
grep -rl 'from "parse"' src
```

It should print `src/services/parse.js` and nothing else. If a component still shows up, it still knows how the database works.

It is not quite the whole story. `Home` never imports Parse and still calls `membership.get("list").get("owner")?.get("username")`, because `App` hands it Parse objects. The to-dos do not have this problem: `toTodo` turns each one into a plain `{ id, text, done, canWrite }` before any component sees it. Doing the same for lists, a `toList` in `listService.js`, is the natural next step, and the one to take when a component needs to know *why* a field might be missing.

Something unclear in this section? Open an issue.

The payoff is a single place to change when the backend does

If Back4App doubles its prices and your group moves to another backend, which files change? Before: every component. After: the files in `services/`. The components ask for `loadTodos(list)` and get an array of plain objects; they do not care who answered.

The more everyday payoff is in the team. One person reshapes the home page, another fixes a sharing bug, and their pull requests touch different files.

With the queries gone, what is left in the components is only the screen, and the next question is whether the screen is cut into the right components: Finding the Components.

Something unclear in this section? Open an issue.

Exam Questions

1. What is a service layer, and what does it take out of a React component?

2. A service function finds that the username does not exist. Why should it throw rather than call `alert`?

3. Your group moves from Parse to another backend. Which files change, if the service layer was pulled out properly? Which change if it was not?

4. How can you check, after a refactoring, that the behaviour of the app did not change?

References

- Martin Fowler — Service Layer — the pattern, from *Patterns of Enterprise Application Architecture*
- Reusing Logic with Custom Hooks — once the service exists, a `useTodos(list)` hook can wrap it together with the state

Code Organization

Source: Lectures/Technical/Structure/Code-Organization.md — something unclear? Open an issue, or send a pull request.

A flat `src/` works until you have to look for things

Here is `todo-26` at `week-08-components`:

```
src/
├── services/
│   ├── parse.js
│   ├── authService.js
│   ├── listService.js
│   └── todoService.js
├── AddForm.jsx
├── App.jsx
├── AuthPage.jsx
├── Home.jsx
├── ListPage.jsx
├── main.jsx
├── MyListRow.jsx
├── SharedListRow.jsx
├── TextInput.jsx
├── ToDoItem.jsx
└── ToDoList.jsx
```

Ten components and `main.jsx` in one folder, sorted alphabetically, which is to say in no order that means anything. `Home` and `TextInput` sit side by side, though one is a whole page and the other is a box around an `<input>`. Nothing in the listing says which is which. You find out by opening them.

For a project one person wrote in a few weeks, that is fine. For a group that will add a dozen screens in the next six weeks, it is not. The services already have a folder of their own, since Pulling Out the Service Layer. The components are next.

Something unclear in this section? Open an issue.

One folder per kind of file: pages, components, services

Sort the files by what they *are*:

- **`pages/`**: one file per screen, the thing a route draws
- **`components/`**: the pieces the pages are built from
- **`services/`**: the code that talks to the backend
- **`utilities/`**: a catch-all, for what you do not know where to put yet

For `todo-26` that is:

```
src/
├── pages/
│   ├── AuthPage.jsx
│   ├── Home.jsx
│   └── ListPage.jsx
├── components/
│   ├── AddForm.jsx
│   ├── TextInput.jsx
│   ├── MyListRow.jsx
│   ├── SharedListRow.jsx
│   └── ToDoList/
│       ├── ToDoList.jsx
│       └── ToDoItem.jsx
├── services/
│   └── …
├── App.jsx
└── main.jsx
```

The test for *page* is simple: does a `<Route>` in `App.jsx` draw it? `Home` and `ListPage` are drawn by routes, and `AuthPage` is drawn in place of all of them. Everything else is a component.

Something unclear in this section? Open an issue.

A component made of several files gets a folder of its own

`ToDoItem` is only ever drawn by `ToDoList`. Nobody else uses it. So the two go in a folder together, `components/ToDoList/`, and the folder says what the file names could not: these belong together, and `ToDoItem` is not meant for anyone else.

`AddForm` and `TextInput` are the opposite case. `AddForm` uses `TextInput`, but so could any form in the app, so `TextInput` stays at the top level of `components/`.

Something unclear in this section? Open an issue.

A file has the name of the component it exports

`ToDoList.jsx` exports `ToDoList`. Always, in this course: one component per file, with the same name, so that searching for a component and opening its file are the same thing. A service file is named after the thing it handles, `listService.js`, and exports functions.

Something unclear in this section? Open an issue.

Moving files is a refactoring too, so do it on a branch of its own

Moving `Home.jsx` into `pages/` changes every `import` that mentions it, and nothing else. That is a perfect pull request: easy to review, and easy to get wrong if it is mixed in with a new feature. Do it on its own branch, as in Collaboration with Git Flow, run the app, and merge.

Your editor helps: in VS Code, dragging a file to another folder in the sidebar offers to update the imports for you.

Something unclear in this section? Open an issue.

No organisation is final

This is a structure to *start* from. When the app grows in a direction it did not foresee, say a dozen components that only the list page uses, give them a folder of their own and move them. You will not get it right the first time, and you do not need to: moving files is cheap, as long as you keep doing it.

Something unclear in this section? Open an issue.

Exam Questions

1. How do you decide whether a component goes in `pages/` or in `components/`?

2. `ToDoItem` lives in a folder with `ToDoList`, but `TextInput` does not live in a folder with `AddForm`. Why?

Component Extraction Cheat Sheet

Source: Lectures/Technical/Structure/Component-Extraction-Cheatsheet.md — something unclear? Open an issue, or send a pull request.

One page, for when you are reviewing a pull request, your own or a teammate's. The reasoning behind it is in Finding the Components, Pulling Out the Service Layer and Code Organization; the examples are all from `todo-26`.

Every pull request is judged against the same five questions

From this week on, this is the review checklist. Before you approve a pull request, you should be able to answer *yes* to all five:

1. **Small components.** Does each component do one job, with everything in it at the same level of abstraction?
2. **State where it belongs.** Is every piece of state in the lowest component that needs it, and no lower?
3. **Clear names.** Do components have nouns for names and handlers verbs, and does every name still fit every place it is used?
4. **Parse behind a service.** Does only `services/` import Parse?
5. **No needless dependencies.** Is every new package in `package.json` worth what it costs? (See Libraries.)

State where it belongs gets its full answer in week 12, with lifting state up. Until then: if two components need the same state, it lives in their closest common parent.

Something unclear in this section? Open an issue.

Extract a block when it repeats, or when you can name it and the page reads better without it

```
Is this JSX the same thing, drawn more than once?
├─ YES → extract it, and name what it draws                  (MyListRow)
└─ NO  → Can you name it in a word or two, as a thing?
         ├─ YES → Would the page read better with just the name?
         │        ├─ YES → extract it
         │        └─ NO  → leave it where it is
         └─ NO  → it is not one thing yet; leave it, or split it
```

When unsure, leave it. Extracting later takes five minutes; a component nobody needed stays forever.

Something unclear in this section? Open an issue.

Four tests catch most bad extractions

Test	Ask	In todo-26
Name	Can you name it as a thing, in a word or two?	`SharedListRow`, not `LeaveButtonAndStuff`
Halves	Would either half be useful on its own? Then it is two.	`AddForm`'s input and button: no, so one
Repeat	Is it drawn more than once?	`MyListRow`, once per list
Level	Is everything in the parent at the same level of abstraction?	`Home` after: headings, rows, a form; no `.get("owner")`

Something unclear in this section? Open an issue.

An extraction moves JSX, and whatever it used from outside becomes a prop

1. **Cut** the block of JSX out of the parent.
2. **List** everything it used and did not define itself: variables from a `map`, the parent's props, handlers. Each one becomes a prop.
3. **Leave the state** where it was, unless only the new component uses it.
4. **Leave the `key`** on the call site, in the `map`; it belongs to the list, not the row.
5. **Click through** the app. The behaviour must not change.

```
// before, in Home
{lists.map((item) => (
  <div key={item.id}>
    <Link to={`/lists/${item.id}`}>{item.get("name")}</Link>
    <AddForm onAdd={(username) => onShare(item, username)} buttonLabel="Share with (username)"
        />
  </div>
))}

// after: item and onShare were used from outside, so they are the props
{lists.map((list) => (
  <MyListRow key={list.id} list={list} onShare={onShare} />
))}
```

Something unclear in this section? Open an issue.

The same mistakes show up in every review

Mistake	Why it hurts	Instead
A component named for an action: `ShareList`	It names the button's job, not the thing on the screen	`MyListRow`, with `handleShare` inside
A name that stopped fitting: `NewTodoForm` used to share lists	The name lies to the next reader	Rename it when its job grows: `AddForm`
`useState` moved into an input wrapper	The parent can no longer read or clear the value	The parent keeps the state; the wrapper takes `value` and `onChange`
`onChange={onChange}` on a wrapped `<input>`	The parent gets an event where it expected text	`onChange={(event) ⇒ onChange(event.target.value)}`
A query in a click handler	The component knows how the database works	A function in `services/`, named in the app's words
`alert` in a service	The service decides how the UI reports errors	The service throws; the component catches and tells the user
The page's heading inside `ToDoList`	The list cannot be drawn without the title, nor the title with anything else	The page draws the heading; `ToDoList` draws the list
A component for every `<div>`	Clutter, for nothing	The stopping rule: extract when it repeats

Something unclear in this section? Open an issue.

For a deeper dive

- React Code Quality: a longer worked example, on a different app
- Semantic JSX: the argument for naming what a block *is*
- Thinking in React

Something unclear in this section? Open an issue.

Finding the Components

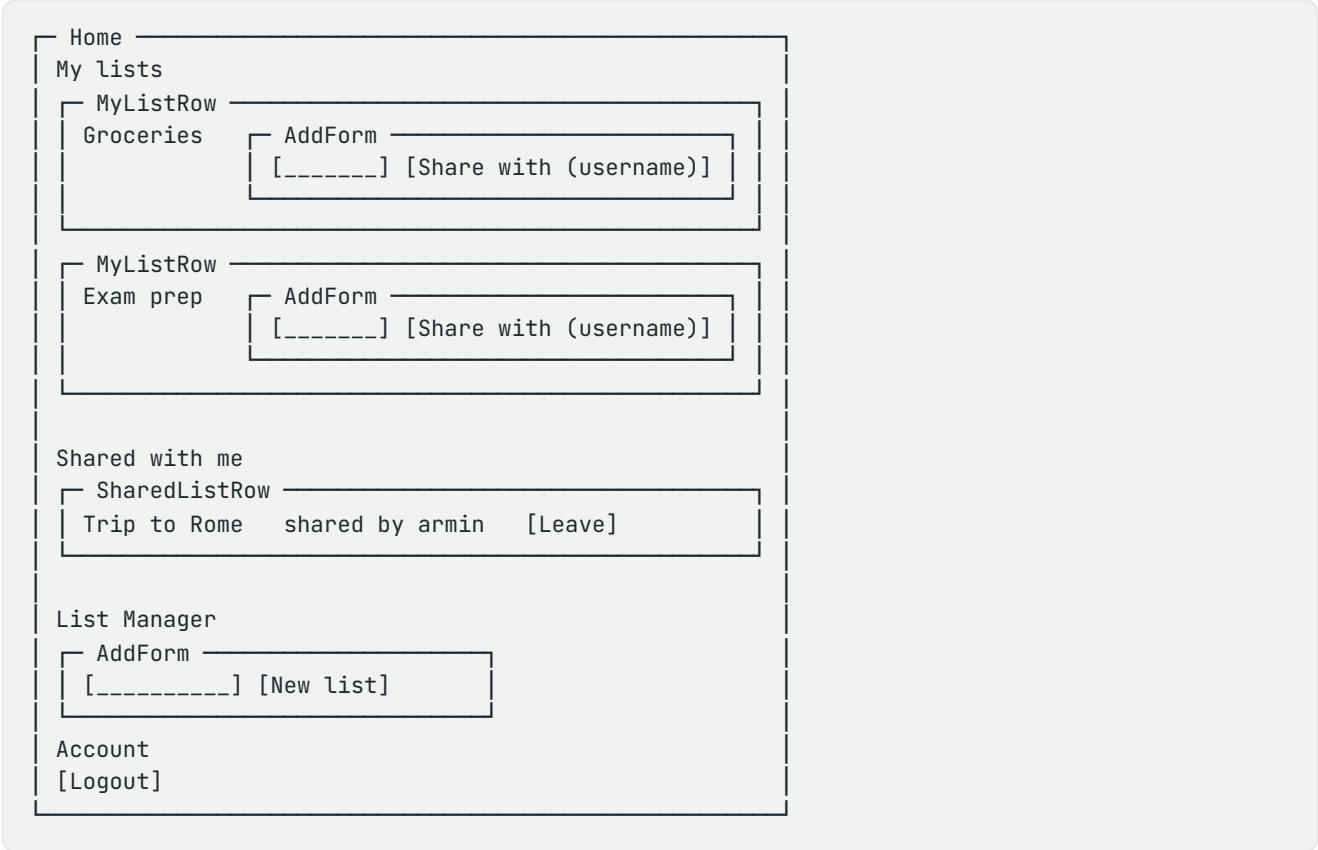
Source: Lectures/Technical/Structure/Finding-the-Components.md — something unclear? Open an issue, or send a pull request.

You know how to write a component. The harder question is which ones to write, and the answer is not obvious from the code, because at that point everything is already one big `return`.

So answer it on the screen instead. Draw boxes around the parts of the running interface, and name each box. It works both ways round: on a screen you have not built yet, and on one that already runs and has grown hard to read.

Start by drawing boxes on the app you can already see

Here is the home page of `todo-26`, as it runs after Pulling Out the Service Layer:



Boxes inside boxes give you the hierarchy for free:

```
Home
├── MyListRow        (one per list I own)
│   └── AddForm      (share it)
│       └── TextInput
├── SharedListRow    (one per list shared with me)
└── AddForm          (a new list)
    └── TextInput
```

The labels are your names for the boxes, not anything the user sees.

Notice what is *not* a box. The *My lists* heading, the *Logout* button and the *Leave* button are drawn but unnamed: each appears in exactly one place. **Draw a box when the same thing appears more than once.** `MyListRow` appears once per list, `SharedListRow` once per membership, and `AddForm` three times on this page alone, and once more on every list's page, for a new to-do.

That is the stopping rule, and you need one, because without it you can keep subdividing until every `<span>` is a component. *My lists* plus the rows under it would make a fine-sounding `MyLists` component, and it would be used exactly once. When you are unsure, leave it where it is: pulling a component out later is a five-minute job, and one you did not need is permanent clutter.

That is the whole exercise, and it takes two minutes on paper.

> **In todo-26:** `git checkout week-08-components`, and compare `Home.jsx` with the one in `week-08-service-layer`. The two rows moved out into `MyListRow.jsx` and `SharedListRow.jsx`; the rest of the page stayed where it was.

Something unclear in this section? Open an issue.

Two boxes if either half would be useful on its own

It is not about how many parts the thing has. `AddForm` is a text input *and* a button, and it is rightly one box: a button with nothing to submit is useless, and a field with no way to submit it is useless. Two parts, one job.

So the test is: **would either half be useful on its own? If yes, make them two. If neither means anything without the other, they are one.**

Then name it as a **thing, not an action**. A component is something on the screen, so it gets a noun: `MyListRow`, `SharedListRow`, `AddForm`. Functions that *do* something get the verb: `handleShare`, `onLeave`. If you find yourself writing `ShareList` as a component, you have named the button's job rather than the thing on the page.

A name also has to fit every place the thing is used. The form began, in Forms and Controlled Components, as `NewTodoForm`. Then it started creating lists, and then sharing them, and `NewTodoForm` with `buttonLabel="Share with (username)"` was a lie. It is `AddForm` now: a form that adds whatever its caller says.

- `MyListRow` — draws one of my lists. One box.

- `SharedListRow` — draws one list somebody shared with me. One box.
- `AddForm` — an input and a button, neither any use without the other. Still one box.

The common way to fail is welding the page's heading into a component that does not need it

`todo-26` still has one. `ToDoList` draws the to-dos and the form to add one, and, at the top, the page's title:

```
export default function ToDoList({ list }) {
  let h1Style = { color: "deeppink", backgroundColor: "white" };
  // …
  return (
    <>
      <h1 style={h1Style}>{list.get("name")}</h1>
      {/* … the to-dos, and the AddForm … */}
    </>
  );
}
```

Want the list without the big pink title, say as a preview of the first few to-dos on the home page? Want a share button next to the title, on the list's page? You cannot have either, because they were glued together before anybody asked.

Both halves would be useful alone, and that is the whole argument. It is the exact opposite of `AddForm`, where neither half would be. Notice too that the name gave nothing away: `ToDoList` sounds like one thing, and it is one thing plus a heading.

The heading belongs to the page. Split, `ListPage` says what goes on it, and `ToDoList` draws only the list:

```
// in ListPage
<>
  <h1>{list.get("name")}</h1>
  <ToDoList key={list.id} list={list} />
</>
```

Two boxes on the drawing, two components in the file. This one is left for you: in `todo-26` it is a two-line pull request.

Two other signals, both visible on the drawing rather than in the code:

It repeats. `MyListRow` appears once per list. Anything you would draw more than once is a component.

It matches the data. Notice that the two kinds of row are the two kinds of thing `App` loads: `lists`, rows of the `List` class, and `memberships`, rows of `ListMember`. One component per kind of row.

When the component hierarchy and the data have the same shape, the code is usually easy; when they disagree, something is being forced. Try drawing one `ListRow` for both, and it fills up with `if (membership)`.

Something unclear in this section? Open an issue.

Extracting a component is moving JSX out and naming what it needs

Here is `Home` with the shared lists still inline:

```
{memberships.map((membership) => (
  <p key={membership.id}>
    <Link to={`/lists/${membership.get("list").id}`}>
      {membership.get("list").get("name")}
    </Link>{" "}
    shared by{" "}
    {membership.get("list").get("owner")?.get("username") ?? "someone"}{" "}
    <button onClick={() => onLeave(membership)}>Leave</button>
  </p>
))}
```

Cut the `<p>` out, and look at what it uses that it did not define itself: `membership`, from the `map`, and `onLeave`, from `Home`'s props. **Everything the moved JSX referenced from outside itself has to become a prop.**

```
export default function SharedListRow({ membership, onLeave }) {
  const list = membership.get("list");

  return (
    <p>
      <Link to={`/lists/${list.id}`}>{list.get("name")}</Link>{" "}
      shared by {list.get("owner")?.get("username") ?? "someone"}{" "}
      <button onClick={() => onLeave(membership)}>Leave</button>
    </p>
  );
}
```

The `key` stays behind. It is for the `map`, which tells one row from the next, not for the row:

```
{memberships.map((membership) => (
  <SharedListRow key={membership.id} membership={membership} onLeave={onLeave} />
))}
```

Once it had a name of its own, `membership.get("list")`, said three times, became one `const`. That kind of small tidying is easier once the code is in a box of its own, where you can see all of it at once.

A component around an input takes what the input takes

The same move works one level down. `AddForm` used to hold its `<input>` inline, as in Forms and Controlled Components:

```
function AddForm({ onAdd, buttonLabel = "Add New Task" }) {
  const [task, setTask] = useState("");
  // …
  return (
    <form onSubmit={onButtonClick}>
      <input type="text" value={task} onChange={(e) => setTask(e.target.value)} />
      <button type="submit" disabled={task.length === 0}>{buttonLabel}</button>
    </form>
  );
}
```

The `<input>` references the value and the change handler, so those become props:

```
export default function TextInput({ value, onChange }) {
  return (
    <input
      type="text"
      value={value}
      onChange={(event) => onChange(event.target.value)}
    />
  );
}
```

One line inside `TextInput` does more than move: `onChange={(event) => onChange(event.target.value)}` unwraps the event and passes up a plain string. Write `onChange={onChange}` instead and the parent gets an event object where it expected text. That is the commonest way an extraction like this goes wrong.

The state stays where it was; `TextInput` has none of its own:

```
<TextInput value={task} onChange={setTask} />
```

`useState` did not move into `TextInput`. If it had, `AddForm` would have no idea what was typed, and could never clear the field or submit the task. `TextInput` displays what it is given and reports what happened; it remembers nothing between renders.

Two things improved, and neither is about saving typing.

The parent stopped touching `event.target.value`. `TextInput` unwraps the event and hands up a plain string, so `AddForm` can pass `setTask` directly. The parent now works in the language of the app instead of the language of the DOM.

There is one place to change how inputs look. A `className`, a placeholder, anything you add later, lives here rather than in every screen that happens to need typing.

And look at the shape of it: `TextInput` takes the value it should show, and a way to report that something changed. Exactly what the raw `<input>` takes, under the same names. The pattern travelled up a level without changing, so the word travels with it: a **controlled component** is any component whose important state is held by its parent, exactly as a controlled input is an input whose value is held by the component around it.

Something unclear in this section? Open an issue.

Exam Questions

1. `AddForm` holds an input and a button and is one component. `ToDoList` holds the list's heading and its to-dos and should be two. What is the difference?

2. What is wrong with a component that renders a to-do heading *and* accepts arbitrary `children`?

3. `TextInput` holds no state of its own. Why not, and what would break if it did?

4. Why does todo-26's home page have a `SharedListRow` component, but no `MyLists` component?

References

- Thinking in React — the five steps, of which the first is the one worth keeping
- Importing and Exporting Components — once a component has its own name it usually wants its own file

Next: deciding *where the state lives* once you have pulled components out is its own problem, and extraction is what creates it: Refactoring by Extracting Components.